University of Calgary

ENSF 614 - Advanced System Analysis and Software Design

- Term Project: Flight Reservation System -

Helder Oliveira, PhD

1. Introduction

The purpose of this project is to practise the complete process of designing and developing a software system using a systematic design methodology.

The final part of the course focuses on system-level design and software architecture. Use all the theoretical concepts and design artefacts discussed during the course as well as your own knoledge as a grad student.

When working in a group, each member is expected to take responsibility for their assigned tasks and make a meaningful contribution. All group members should participate in lectures, lab sessions, and group discussions, and complete their assigned work to the best of their ability.

2. Project Overview: Flight Reservation System

Design and develop a standalone Flight Reservation System using object-oriented principles and domain-driven design concepts. The primary focus is on domain analysis and design, including class modelling, associations, use-case definitions, and the application of suitable design patterns.

The application will simulate a flight-booking system through a single-user, role based, graphical user interface (GUI). It is not intended to be a web-based or client-server application.

The system should allow a user, acting as a customer, flight agent, or system administrator, to:

- View available flights and flight details.
- Create, cancel, or modify reservations.
- View booking history and flight schedules.
- Manage flight information (add, update, and delete flights).
- Maintain customer information and payment records.

2.1 User Roles

Role	Description	Key capabilities
Customer	End user who books flights	Search for flights; create and cancel bookings; view reservations
Flight agent	Employee who manages bookings	Manage customer data; modify reservations; view schedules

Role	Description	Key capabilities
System administrator	User who administers system data	Add and remove flights; manage routes and aircraft; update schedules

3. Functional Requirements

The following are the main functional requirements. Teams may identify and document additional requirements where appropriate. For example, some features, such as the promotions, will need to use a random component to select the promotion to be shown to the customer.

3.1 Flight Search and Viewing

- Search for flights by origin, destination, date, and airline.
- Display available seats, flight times, and prices.

3.2 Booking Management

- Create new bookings.
- Cancel or modify existing reservations.
- Generate booking confirmations.

3.3 Customer Management

- Create customer profiles.
- View and edit customer details.

3.4 Flight Management (Administrator Only)

- Add flights and update existing flight details.
- Remove outdated flights.

3.5 Payment Simulation

- Record payment information. Payment processing is simulated; no real payment processing is required.

3.6 Database Connectivity

- Use a local or embedded database, such as MySQL or SQLite, to store flight and reservation data.

3.7 Monthly Promotional News

- Notify customers about monthly promotions on the first day of each month.
  - A promotion is randomly selected when the application is first opened during a given month, and is shown only once that month.

4. Non-Functional Requirements

The system should address the following non-functional requirements:

- **Usability:** Provide a simple GUI using Java or C++.
- **Portability:** Run as a standalone application on a desktop system.
- **Maintainability:** Use a clear modular design and a layered architecture.
- **Scalability:** Allow for future extensions, such as online booking or a client-server architecture.
- **Additional requirements:** Teams may identify other relevant non-functional requirements.

5. Recommended System Architecture

A three-layer architecture is recommended:

1. **Presentation layer:** GUI and input handling.
2. **Business logic layer:** Domain classes and application operations.
3. **Data layer:** Database and file-access functionality.

6. Assessment Criteria

The project will be evaluated on its functionality and the quality of its design, including:

1. Separation of concerns across software architecture components (e.g., entities, controllers, and views).
2. Application of design-quality principles, including modularity, cohesion, and low coupling.
3. Treatment of non-functional requirements, including usability, reusability, maintainability, and reliability.
4. Separation of concerns across system levels: presentation (view/boundary classes), domain (entity classes), and control classes that coordinate them.
5. Clarity and accuracy of the UML diagrams and how well they represent the final application.
6. Coverage of the project requirements in the use-case diagram and traceability from use cases to the class diagram.
7. Quality of the use-case scenarios and sequence diagrams used to describe the system's behaviour.

7. Expected Deliverables

The project has two phases: design (50 marks) and implementation (50 marks).

7.1 Design Phase (50 Marks)

Submit a design document, in PDF format, containing a clear description of the system requirements and the following design artefacts.

A. System Introduction and Functional Processes

Introduce the system and provide a brief description of its purpose. Then describe several major or relatively complex processes, such as searching for and selecting a flight, booking a ticket, and

making a payment.

Include the following four activity diagrams:

1. Login process.
2. Browsing for and selecting a flight.
3. Booking a flight.
4. Making a payment.

B. System Use-Case Diagram

Provide a use-case diagram for the system.

C. Use-Case Scenarios

Write a detailed scenario for **three use cases** of your choice, shown in the use-case diagram. Underline candidate classes with a single underline and candidate operations with a double underline.

D. Sequence Diagrams

Provide **four sequence diagrams** for **four major use cases**. Each use case must be represented by a control class in the domain class diagram.

E. State-Transition Diagrams

Provide three state-transition diagrams:

1. One representing the state of the overall system.
2. One representing the payment process.
3. One representing a reservation object.

F. Class Diagram

Provide a class diagram showing classes, relationships, attributes, behaviours, and cardinalities.

Class-diagram notes:

- The classes in the diagram must be traceable to the use-case scenarios. Marks may be deducted for classes that cannot be traced to the use cases.
- You do not need to include Java/C++ library classes such as exceptions, buttons, string, or any data structure.
- Clearly identify class stereotypes, such as `<<entity>>`, `<<control>>`, and `<<boundary>>`.
- Constructors, destructors, getters, and setters do not need to be shown.
- At the final stage, refine the architecture to improve reusability, scalability, maintainability, and reliability. Use appropriate concepts as needed, including modular design, inheritance, realization, aggregation, composition, polymorphism, and design patterns.

G. Package Diagram

Provide a package diagram showing how the three architectural layers interact. Show the classes in each package and identify which classes are public.

7.2 Implementation Phase (50 Marks)

Implement the proposed design as a Java or C++ GUI application using the GUI library of your choice, with MySQL or SQLite for data storage.

Submit a ZIP archive containing:

1. A compiled file (JAR or executable).
2. A ZIP archive containing all `.java` or `.cpp/.h` source files.